# THE UNIVERSITY OF AZAD JAMMU & KASHMIR, MUZAFFARABAD

| | |
|---|---|
| COURSE TITLE | OOP |
| COURSE CODE | CS-1204 |
| TITLE | PROJECT REPORT |
| PROJECT NAME | GAMING SYSTEM |
| GROUP NO. | 04 |
| INSTRUCTOR NAME | ENGR. AWAIS RATHORE |
| SUBMISSION DATE | SEPTEMBER 27, 2025 |

## DEPARTMENT OF SOFTWARE ENGINEERING

# Contents


Introduction ..... 3
Objectives ..... 3
Features ..... 3
Games Overview ..... 4
Module Descriptions ..... 4
File Handling ..... 5
Sample Execution Flow ..... 5
Strengths of the Project ..... 5
System Architecture ..... 6
- Class Diagram ..... 6
- Flow Chart ..... 7

Possible Improvements ..... 8
Conclusion ..... 8
Submission ..... 8
- GitHub Repository ..... 8


# Group # 4

## Members:


Haseeb Sheikh 2024-SE-31

Zoya Mazhar 2024-SE-26

Arsalan Awan 2024-SE-28

# OOP

# Project Report: Gaming System

## Introduction

The Gaming System project is a C++-based application designed to provide users with multiple interactive mini-games while maintaining player profiles, scores, and achievements. It demonstrates fundamental Object-Oriented Programming (OOP) principles, file handling, and modular design using header files. The system is built to be easily extendable, allowing more games to be integrated seamlessly.

## Objectives

| Objective | Description |
|---|---|
| Modular Gaming System | Provide multiple games under one platform using C++ |
| OOP Implementation | Use inheritance, polymorphism, and encapsulation |
| Player Profiles | Save player stats like name, score, and coins |
| Variety of Games | Offer quizzes and interactive games |
| User-Friendly | Use retry mechanisms, input validation, and color formatting |

## Features

| Feature | Description |
|---|---|
| Player Profile Management | Stores player details and updates them in scores.txt |
| Authentication | Uses passphrase.txt for login validation |
| Games Included | Number Guess, Tic Tac Toe, Time Travel Quiz, Code Debugger, GK Quiz |
| Color Output | ANSI escape codes for colored console interaction |
| Retry Mechanism | Players can replay games without restarting |

## Games Overview

| Game | Description | Scoring/Reward |
|---|---|---|
| Number Guess | Player guesses random numbers in a limited tries | Points based on attempts left |
| Tic Tac Toe | Classic two-player board game | Win/draw adds points |
| Time Travel Quiz | Quiz on historical events | Correct answers add score |
| Code Debugger | Debugging coding errors | Points for identifying mistakes |
| General Knowledge Quiz | Mix of general and university-specific questions | Points for correct answers |

## Module Descriptions

| Module | Function |
|---|---|
| Main.cpp | Entry point, handles setup, authentication, and menu |
| Player.h | Defines Player class, manages profile and persistence |
| Game.h | Abstract base class with play() and retry mechanism |
| NumberGuess.h | Implements number guessing game |
| TicTacToe.h | Implements Tic Tac Toe logic |
| TimeTravelQuiz.h | Implements history-based quiz |
| CodeDebugger.h | Programming/debugging challenge |
| GKQuiz.h | General knowledge quiz |

## File Handling

| File | Purpose |
|---|---|
| passphrase.txt | Stores authentication key |
| scores.txt | Saves player profile and stats |

## Sample Execution Flow

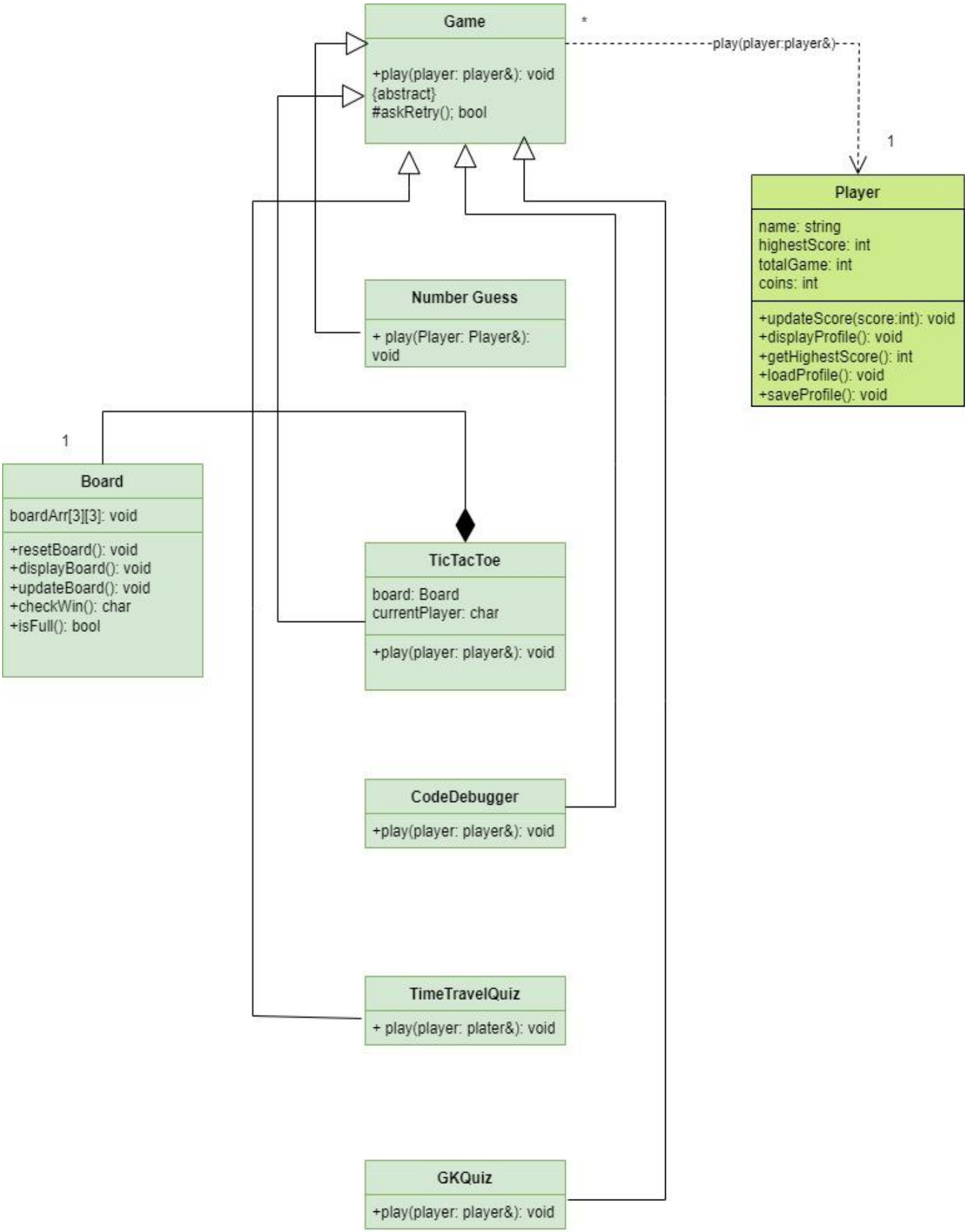
| Step | Action |
|---|---|
| 1 | Program checks passphrase.txt |
| 2 | Player logs in and loads profile from scores.txt |
| 3 | Player selects a game from the menu |
| 4 | Game executes and updates player stats |
| 5 | Player can retry or select another game |

## Strengths of the Project

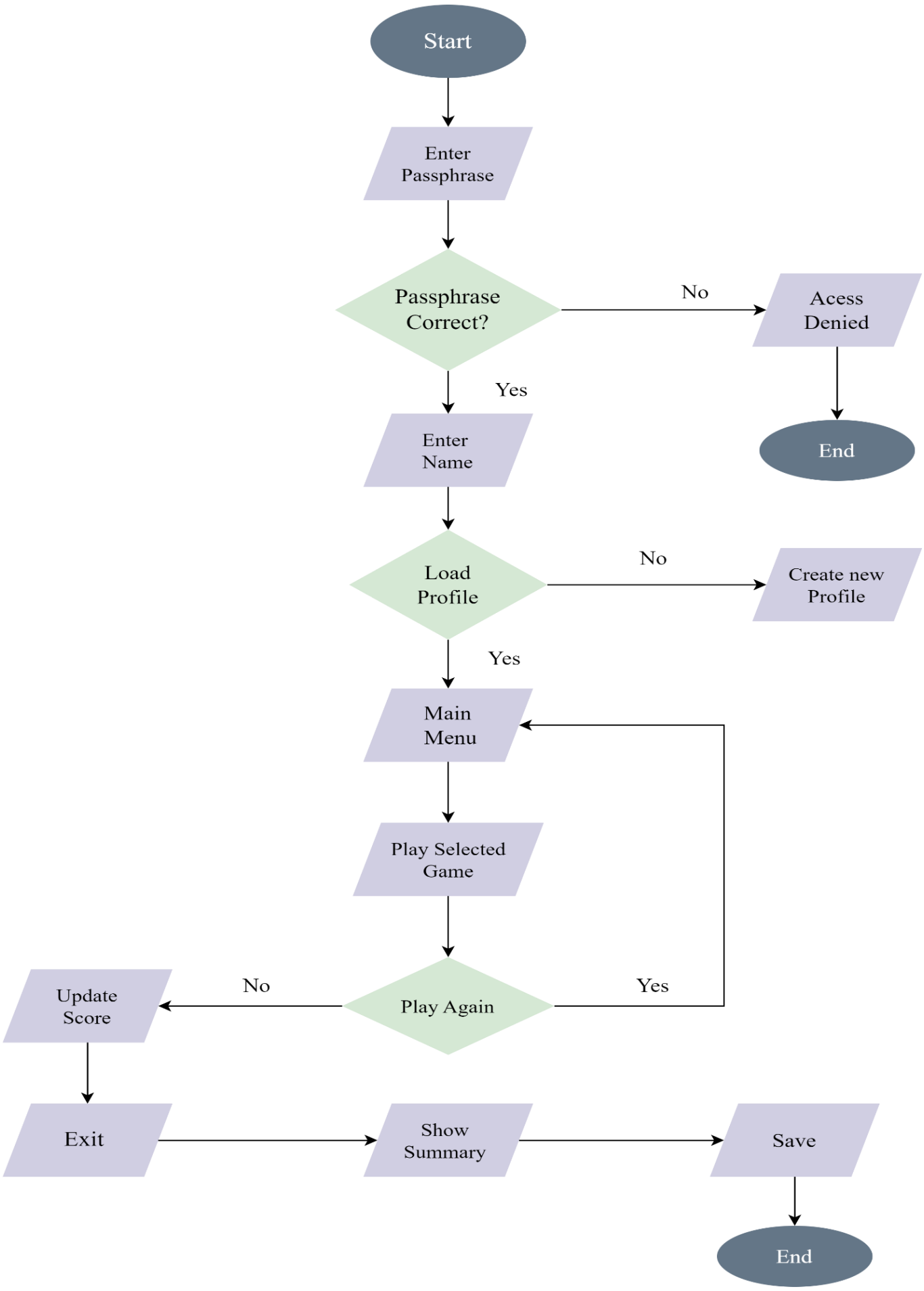
| Strength | Description |
|---|---|
| Extensibility | Easy to add new games via inheritance |
| Persistence | Player data saved between sessions |
| Variety | Multiple games keep players engaged |
| Educational | Includes quizzes that promote knowledge |

# System Architecture

## Class Diagram



# Flow Chart


Start
Enter Passphrase
Passphrase Correct?
No
Acess Denied
End
Yes
Enter Name
Load Profile
No
Create new Profile
Yes
Main Menu
Play Selected Game
Play Again
No
Update Score
Yes
Exit
Show Summary
Save
End

# Possible Improvements

| Improvement | Benefit |
|---|---|
| Add menu system | Easier navigation between games |
| Multiplayer support | Enable competitive play |
| Use database (SQLite) | Better data management than text files |
| Difficulty levels for quizzes | Make gameplay more adaptive |
| Graphical interface (SFML) | Enhance UI/UX visually |

# Conclusion

The Gaming System project successfully demonstrates how C++ can be used to create a modular, extensible gaming platform. It combines entertainment, learning, and technical demonstration of OOP concepts. With further improvements, it can evolve into a more advanced system with greater interactivity and richer features.

# Submission

This was a Group Project, so each member did some work and uploaded it on the GitHub. I have included the link of the GitHub repository, where the whole project can be easily accessed. Each member uploaded the work he did, which you can easily find out by the commit history.

**GitHub Repository:** https://github.com/haseeb-sheikh1/OOP-Project